CLIP architecture for LLM-based RNA-protein exploration on cite-seq data

Project proposal for the
Lemanic Life Science Hackathon 2026

Anna Seres, Teo Secilmis, Antonin Thiébaut, Bernd Illing

Gene translation

- Biological process that produces protein from RNA
  https://en.wikipedia.org/wiki/Translation_(biology)
- In principle there is causal link from RNA -> protein, however:
  - More than one gene (RNA) can influence translation
  - Process is not linear
  - Noisy, internal noise and external (unknown) factors
  -
- -> Modeling and predicting protein expression from RNA is difficult
- This is where machine learning can come in useful, especially if there is paired RNA (input) and protein (target) data as in cite-seq.

Classic ML prediction models

- Predictive models (RNA -> protein) on citeseq
- Important considerations:
  - Input feature (RNA) normalisation/selection/filtering/representation. Raw data are counts (per cell). Common steps are normalisation (counts per cell, log1p) highly variable genes, PCA etc, see e.g. https://scanpy.readthedocs.io/en/stable/tutorials/basics/clustering.html
  - Target (Protein) processing. Raw data are counts (per cell). Common are variance-stabilizing transforms, ADTNorm and binarisation/thresholding (i.e. only keeping the information whether or not a protein is expressed in a cell).
    - Important: the nature of the (processed) target, i.e. integer counts, continuous normalised counts, or binary variable dictates the model and loss choice, i.e. regression vs. classification.
  - Model choice: anything possible from linear regression to decision trees to transformers…

RNA (all genes or subset)

Protein (only one or multiple)

Classic ML prediction models

- Predictive models (RNA -> protein) on citeseq have been explored in Neurips challenge/kaggle competition https://openproblems.bio/events/2022-08_neurips
- Summary paper: Longitudinal modality prediction learns gene regulatory patterns: insights from a single-cell competition
- Conceptually simple and good starting point for our hackathon -> provides baseline performance, i.e. what to expect in terms of predictability of protein from RNA
- Limitations:
  - Fixed application: only prediction possible, no further querying or RNA data
  - Not flexible in terms of panel: not possible to predict proteins that weren't in train panel
  - Non-intuitive interface for non-bioinformaticians: output is vector of (normalized?) protein counts for a fixed panel

CLIP model as an exploratory extension of protein prediction models

- We want flexibility in terms of application
- We want to be flexible in terms of panel: only relevant proteins should be predicted, “new” proteins (i.e. not part of training set) should be predictable
- More intuitive user interface: input preferably natural language queries from user. Output will be a ranking of queries.

CLIP https://arxiv.org/pdf/2103.00020

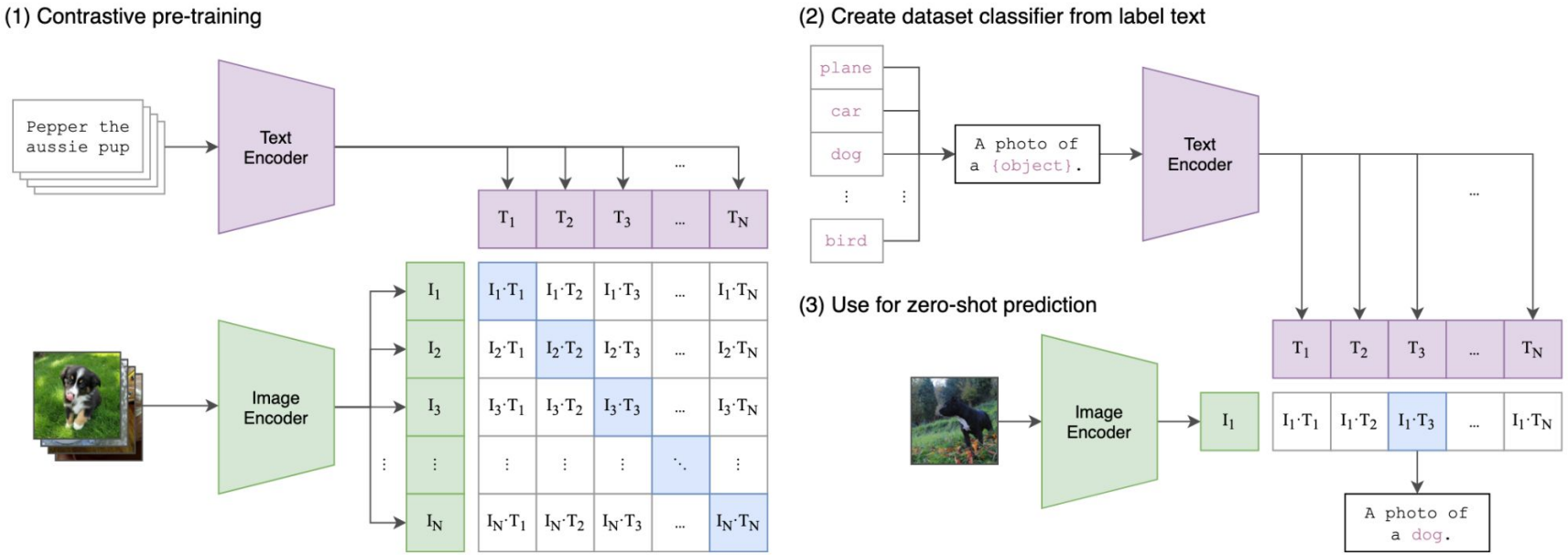
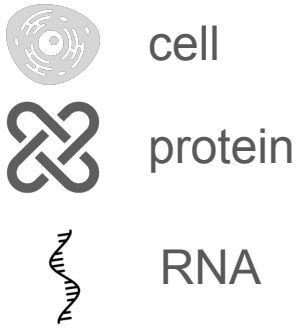
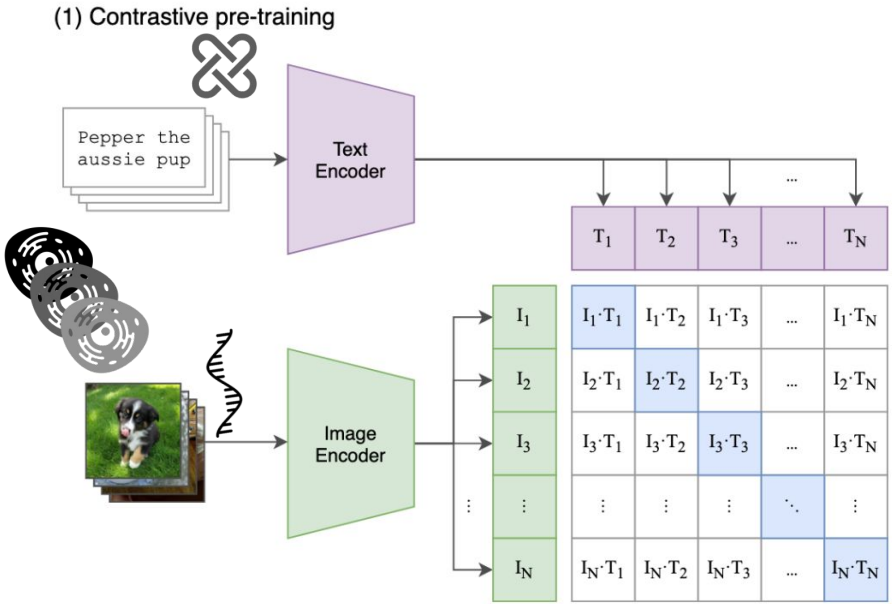
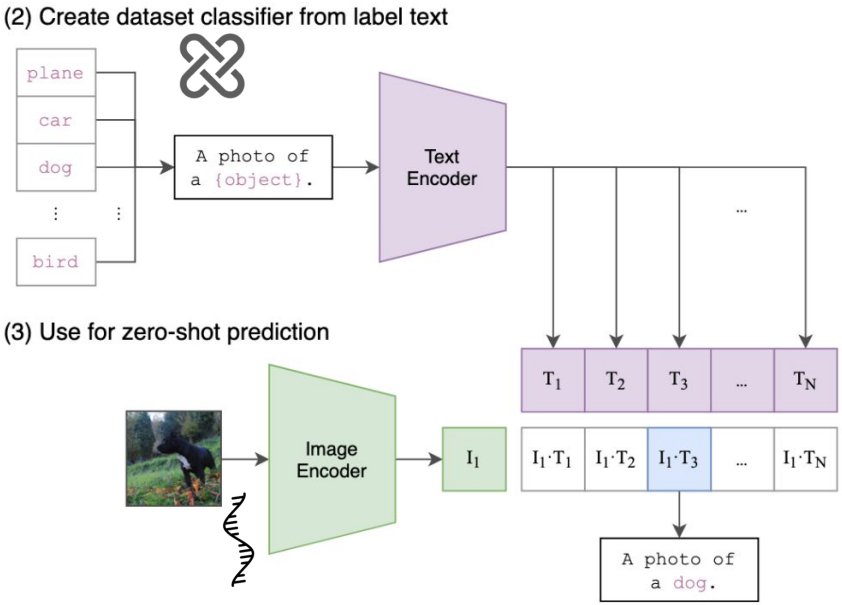


Figure 1. Summary of our approach. While standard image models jointly train an image feature extractor and a linear classifier to predict some label, CLIP jointly trains an image encoder and a text encoder to predict the correct pairings of a batch of (image, text) training examples. At test time the learned text encoder synthesizes a zero-shot linear classifier by embedding the names or descriptions of the target dataset's classes.

CLIP for cite-seq data







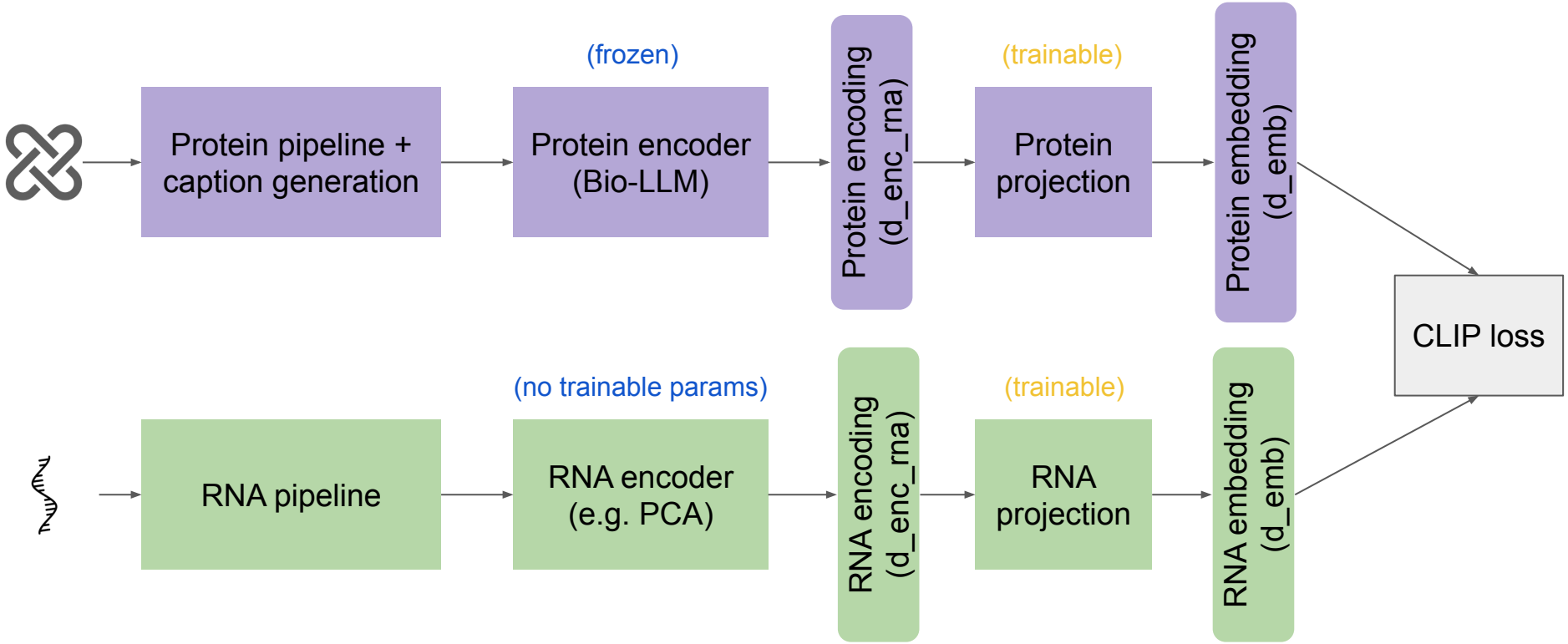
More concretely

- RNA encoder (in place of image encoder):
  - Standard feature selection + normalisation pipeline
  - Dimensionality reduction using PCA
  - Optional: encode RNA using a more fancy method, e.g.
    - scVI: common deep model for integration, fitted on the current data set
    - single cell foundation model like scGPT (helical could be useful for that https://github.com/helicalAI/helical): pre-trained model leveraging large corpus of data, allowing flexible amount/subset of genes as input
  - -> resulting vector is the RNA encoding
- Protein encoder (in place of text encoder)
  - ADTNorm pipeline + thresholding -> binary protein expression per cell
  - Creation of “cell captions” as natural language (strings) describing the (binary) protein expression. Should be sampled/generated randomly picking a random number of expressed or non-expressed proteins. Can include the cell type label if available. E.g. “This (... T-cell) expresses protein A, B and C. This cell does not express protein X and Y.”
  - Embed those caption strings with a (pretrained, frozen) bio-informed LLM, e.g. BioBert.
  - -> resulting vector is the Protein encoding

From encoding to (common) embedding

- So far, the RNA encoding and Protein encoding are frozen and hence not trainable or adaptive and might have different dimensionality.
- For CLIP training we need (i) trainable embeddings (ii) of the same dimensionality.
- We achieve this by passing the encoding through respective RNA and protein projection networks. Let's choose a simple MLP (e.g. or even a linear layer for starters), e.g.:
  - RNA: d_enc_rna -> d_hidden -> ReLu -> d_emb
  - Protein: d_enc_protein -> d_hidden -> ReLU -> d_emb

Architecture sketch



Training

- Split citeseq dataset into train, eval and test.
  - Split along cells (batches/diseases). Make sure to split grouped by donors/batches and potentially stratified by cell types/proteins.
  - Split along proteins: keep some proteins out to test “zero-shot” capabilities on “unseen” proteins
- Use citeseq RNA & protein as input to train projection networks using CLIP loss.
- Sample different “cell captions” to train robust embeddings.
  - Change the number of included protein markers in the “captions”
  - Try only including positive markers
  - Try out different orders of markers or make sure the “captions” are more structured, such that the LLM understands that it is a set of markers, not an ordered list
- Keep LLM fixed for now in order to not “overwrite” previously learned biological knowledge that could help in future zero-shot inference.
- Performance consideration: it probably makes sense to cache the protein encodings (i.e. caption string embeddings, i.e. the output of the frozen LLM) and directly train on those embeddings in order to avoid excessive LLM forward calls. Make sure to cache multiple captions per cell as described above.

Inference + evaluation

- Inference works equivalently as for the original CLIP: we input the RNA as usual with the RNA branch of the model and obtain an embedding.
- We then compile a list of query protein cell captions, e.g. [“This cell expresses protein A and B”, “This cell expresses protein A but not B”, etc.] and encode each query with the protein branch of the model, giving us an embedding per query.
- Then each query embedding is compared against the RNA embedding (as in CLIP training), and we obtain a ranking of the queries. The highest scoring query is the “prediction” of the model.

Further exploration & analysis

- Once model predictions are obtained as described before, those predictions can be compared to a traditional ML predictive model
- However, we have full flexibility with which “caption” to query:
  - We can include new, unseen proteins and query for their expression. This assumes that the bio LLM does a good job embedding the new query and that the projection networks learned a good shared embedding space.
  - We can test different queries, e.g. including the cell type, reformulating the query etc. This is similar to prompt engineering for generative LLMs.
- We can also flip it: given a protein query, which cells in a given population (dataset) match this query (the best). This can help identify subpopulations. Since we have cell type annotations we can compare to ground truth.
- The learned (shared) embedding space can be investigated through clustering, visualization etc. using e.g. scanpy

IMPORTANT: Useful starting point for implementation:

- There are minimal implementation of CLIP models, e.g. https://github.com/filipbasara0/simple-clip/tree/main.
- It is encouraged to use one of those. As is or at least for inspiration.
- The core architecture, loss, train- and inference code can be directly adopted
- Key things to change:
  - Architecture:
    - The vision encoder part needs to be changed into an RNA encoding part
    - The language model has to be adapted (to e.g. BioBert). This could be a one-liner just changing the model name.
  - Dataset/Dataloader: The dataset needs to be adapted:
    - Images -> RNA encodings
    - Textual image descriptions -> textual protein descriptions/queries
    - Optional: implement data augmentations (“views of a cell”), e.g. RNA dropout, varied protein queries etc. …

Further links

- Paper with a similar CLIP idea, but with RNA-text pairs: https://cellwhisperer.bocklab.org/
- Citeseq data https://en.wikipedia.org/wiki/CITE-Seq
- ADTNorm paper https://www.nature.com/articles/s41467-025-61023-6
- Anndata https://anndata.readthedocs.io/en/stable/ and muon https://muon.readthedocs.io/en/stable/api/generated/muon.MuData.html data structure libraries
- Scanpy single cell library for processing, clustering and visualisation https://scanpy.readthedocs.io/en/stable/
- Helical https://github.com/helicalAI/helical for single cell foundation models
- Sklearn for traditional machine learning, data splitting, scoring etc https://scikit-learn.org/stable/index.html
- Minimal clip implementation as a starting point, same as previous slide: https://github.com/filipbasara0/simple-clip/tree/main